The Accessibility Police

An Automated Website Accessibility Checker

Project Description

The purpose of the program is to analyze a webpage and identify common accessibility issues, such as missing form labels and missing alt text. The user will enter a website URL, and the program will retrieve and analyze the page's HTML. The program will then check for common and identifiable accessibility issues, including missing or empty page titles, missing HTML language information, missing alt text for images, forms without associated labels, links without discernible text, and problematic heading structures. Instead of just displaying technical error messages, the goal of the program is to explain to the user why each issue matters from an HCI perspective. For example, instead of saying that an image is missing an alt attribute, the program could explain that a person needing a screen reader wouldn't be receiving important information about the image. Each issue will include a severity level, a description of the problem, and an explanation of its potential effect on users.

The program uses several Python packages to retrieve and analyze websites. It uses the requests package to retrieve the website from the provided URL, and BeautifulSoup to parse the HTML and identify the different elements, like images, headings, forms, and tables. Depending on which accessibility checks I'm able to implement, I may also use additional packages to help analyze things like color contrast. The result of each check will be stored as structured data, containing the WCAG guidelines, the severity level, the affected HTML element, the description of the problem, and the explanation of its potential impact on users. The program will be intended as a "first pass" automated accessibility checker, rather than a complete professional accessibility audit. More complex checks, like color contrast, may be added later if I have time.

For the first version, the program will run entirely through a CLI. The user will enter a URL, and the program will then retrieve the website, perform the accessibility checks, and print a summary of the issues it found. For example, the output would show the number of high, medium, and low severity issues, followed by a description of each issue and why it causes an issue for users. Starting with CLI will allow me to focus first on making the website analysis and accessibility checks work correctly before spending time on a GUI.

For the GUI, built with Streamlit, the user will enter the URL into a text field and click a button to analyze the website. The results would be displayed in a dashboard showing an overall accessibility summary, severity levels, categories of issues, relevant WCAG guidelines, and detailed explanations for individual problems.

User Flows

1. Enter website URL

A user wants to check whether their website has common accessibility problems. They open The Accessibility Police accessibility checker and receive a prompt to enter a website URL that they want analyzed. For the initial version of the program, the user enters the URL through a CLI and submits it. In the web-based version, the user will enter a URL into a text field and click on a button to start the analysis. The program checks that the URL is in the correct format, and then prepares to retrieve the website HTML.

Once the URL has been submitted, the program sends a request to the website and retrieves the website's HTML. If the webpage can't be reached, the program will display an error message explaining that the page couldn't be accessed.

Technical Details

- v1 will accept the URL through a CLI.
- The eventual GUI will use a text field for input and a button to start the analysis.
- The requests package will retrieve the website.
- The program will check the HTTP response to determine if the page was retrieved successfully.
- BeautifulSoup will be used to parse the returned website's HTML.
- Invalid URLs or unsuccessful responses will be handled with an appropriate error message.
- The retrieved HTML will become the primary data source for the accessibility analysis.

2. Run the accessibility analysis

After the website has been successfully retrieved, the accessibility analysis begins. The program examines the website's HTML and performs a series of automated checks. These checks will look for common accessibility problems that can be identified from the page's structure and code. As the program performs each check, it records the results. Problems are categorized by WCAG guidelines. The program also keeps track of checks that pass so that the user receives an overall picture of the website's accessibility rather than just a list of errors.

Technical Details

- BeautifulSoup will be used to locate HTML elements, like <img>, <input>, <label>, <a>, <h1…h6>, and <html>.
- Results will be stored as structured data, like a dictionary or list.
- Each identified issue will include: WCAG guideline, severity, description, affected HTML element, and explanation of the potential user impact.
- Some checks, such as color contrast, may be implemented as a later feature because the information required is not available from HTML alone.

3. Review and understand the results

Once the analysis is completed, the user can review the results to understand the accessibility problems that were found on the website. The program first gives a summary of the results, like the number of issues found at each severity level. The user can then look at the individual findings to see what is wrong with the website and why it matters. The goal of this task is to make the results understandable to anyone, not just people with a technical background. Instead of expecting users to understand HTML terms and accessibility guidelines, the program will translate each technical finding into plain language and, when possible, offer general suggestions for resolving the issues.

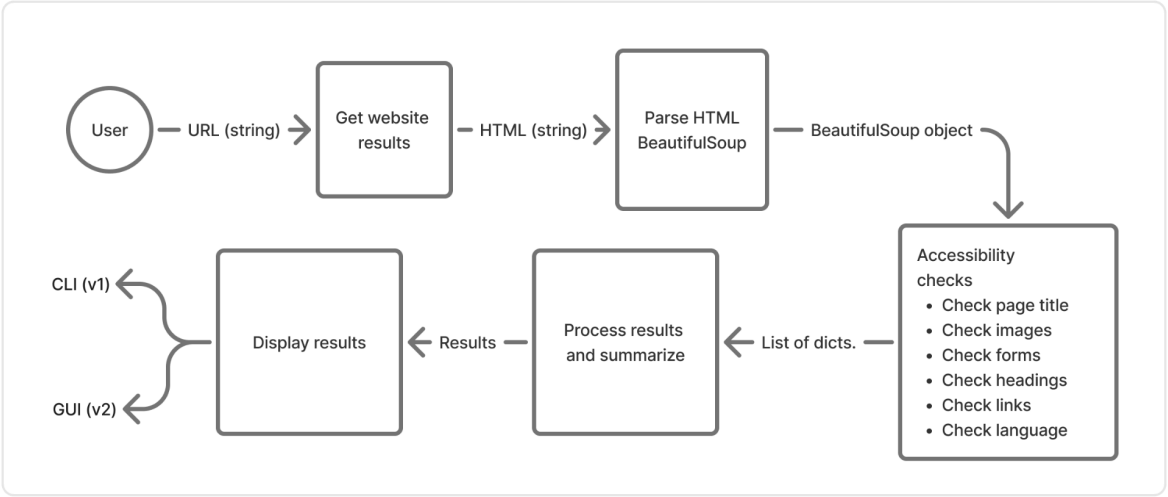
Technical Details

- Results will initially be displayed through the CLI.
- Results will be grouped by severity: high, medium, and low.
- The program will display a summary before showing individual issues.
- Each issue will contain an explanation in plain language of its impact on users
- The program will provide a recommendation to fix, or a next step
- The output will be generated from the structured results that were produced during the analysis step.

Technical Flow

Data Flow Diagram

Data Flow Diagram



Data Types

Block	Input	Output
User Input	URL (str)	URL (str)
Get website	URL (str)	HTML (str)
Parse HTML	HTML (str)	BeautifulSoup object
Accessibility checks	BeautifulSoup object	List of issue dicts.
Process results	List of dicts.	Summary dict.
Display results	Summary + issue list	CLI/GUI output

Individual Accessibility Issue Example

```
{
    “type”: “Missing alt text”,
    “severity”: “High”,
    “element”: “<img src=’header.jpg’>”,
    “description”: “Image doesn’t have alternative text.”
    “user_impact”: “Screen reader users may not get the information conveyed in the image.”
}
```

Possible Function Structure

```
def get_webpage(url):
    # Retrieve website

def parse_html(html):
    # Parse HTML with BeautifulSoup

def check_page_title (soup):
    # Checks whether the website has a non-empty <title> (WCAG 2.4.2)

def check_images(soup):
    # Find images missing alt text (WCAG 1.1.1)

def check_forms(soup):
    # Find form inputs without labels (WCAG 1.3.1, 3.3.2)
```

```
def check_headings(soup):
    # Check heading structure (WCAG 1.3.1)

def check_links(soup):
    # Check for empty or unclear links (WCAG 2.4.4)

def check_language(soup):
    # Check for missing language attribute (WCAG 3.1.1)

def analyze_page(soup):
    # Run all accessibility checks

def display_results(results):
    # Display findings through CLI
```

Nice-to-Have Features

The following features may be explored if time allows after the core functionality and reliability of the existing checks has been addressed:

- **Additional accessibility checks**: Implement checks for buttons without accessible names, iframes without descriptive titles, duplicate HTML IDs, and invalid form-label associations.
- **Color contrast analysis**: Look for ways to identify potential color contrast issues; this may require additional tools and more advanced analysis than my current approach.
- **Explore fast-a11y-py**: Could offer a way to evaluate additional accessibility rules.
- **Improved guidance**: Provide more specific suggestions for resolving detected issues, potentially including examples of corrected HTML.
- **Export**: Allow users to export their results to a file for review.

Final Assessment

The biggest change I made from the original sketch was narrowing down the types of accessibility issues that the program will be checking for. I decided to focus the initial version on issues that can be more reliably identified from a website's HTML, like missing alt text, missing form labels, heading structure, and missing language information. This makes my project more realistic while leaving room for me to add more advanced checks later if I have time.

I'm fairly confident that I can implement my project as it's currently written. I'm comfortable with the general Python concepts that'll be needed, and I think most of the individual problems can be solved through research as I work through them. The biggest potential problem is making sure the accessibility checks produce useful/accurate results instead of incorrectly identifying something as an accessibility issue. I'll need to carefully test the program with different sites and account for situations where an automated check might not be able to determine whether something is actually accessible.

The parts I'm least familiar with are the GUI options. I haven't worked with Streamlit or Flask before, so I may need help understanding how to take Python functionality I build for the CLI and connect it to a web-based interface. However, I'm comfortable learning new Python libraries through docs and examples, so I don't think the GUI will prevent me from completing the core project.

Overall, I feel very confident that I can do the core functionality of the project independently. I expect that I can work through most of the technical challenges outside of Streamlit/Flask by using documentation, examples, and Google. The main area where I might need assistance is determining the best approach for implementing the web-based interface and potentially figuring out how to handle the more complex accessibility checks.